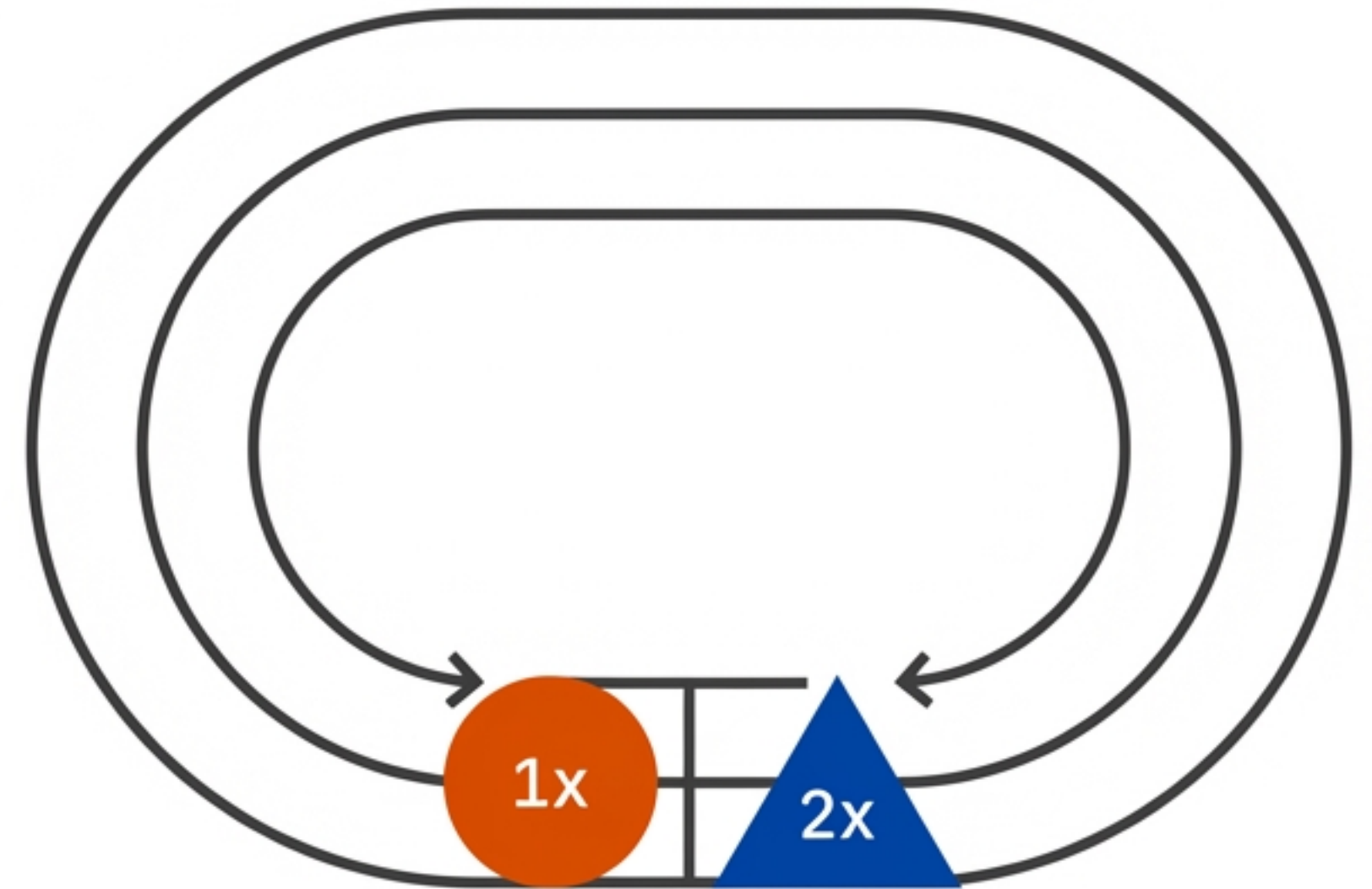


Mastery of the Fast & Slow Pointers Pattern

Day 9/90 DSA Course | Solving 3 Popular Interview Questions

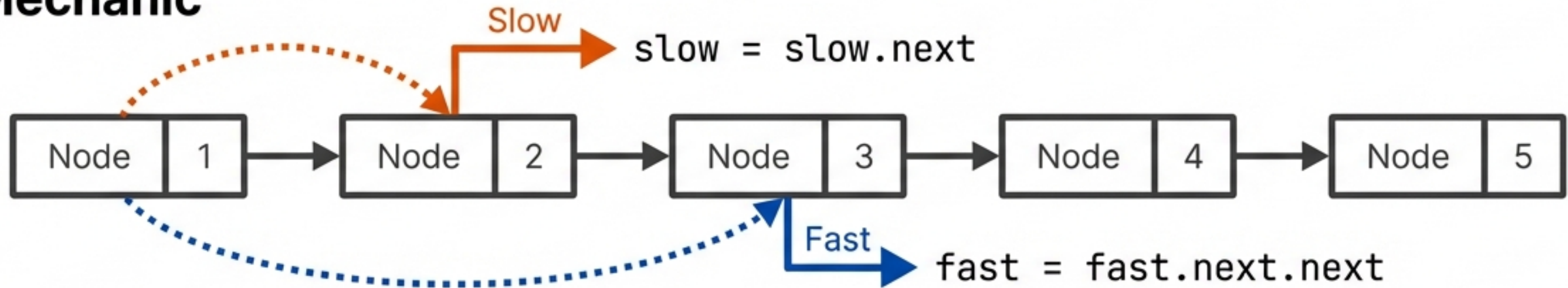


Based on the teachings of CTO Bhaiya.
Validated for DSA 2025.

The Intuition

Why use two pointers at different speeds?

The Mechanic

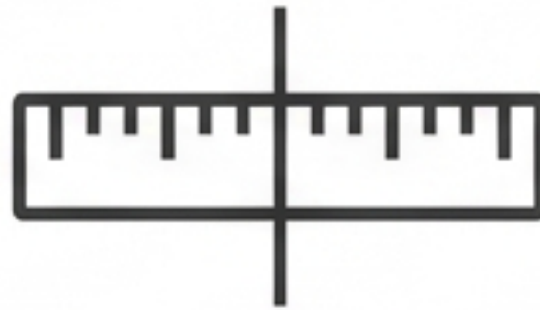


The Utility



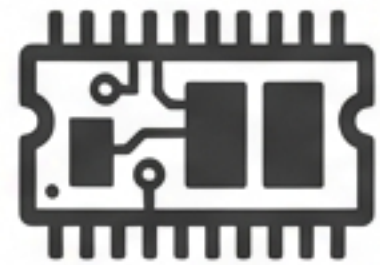
Cycle Detection

If a loop exists, Fast will eventually lap and collide with Slow.



Finding Midpoints

When Fast reaches the end, Slow is exactly at the middle.



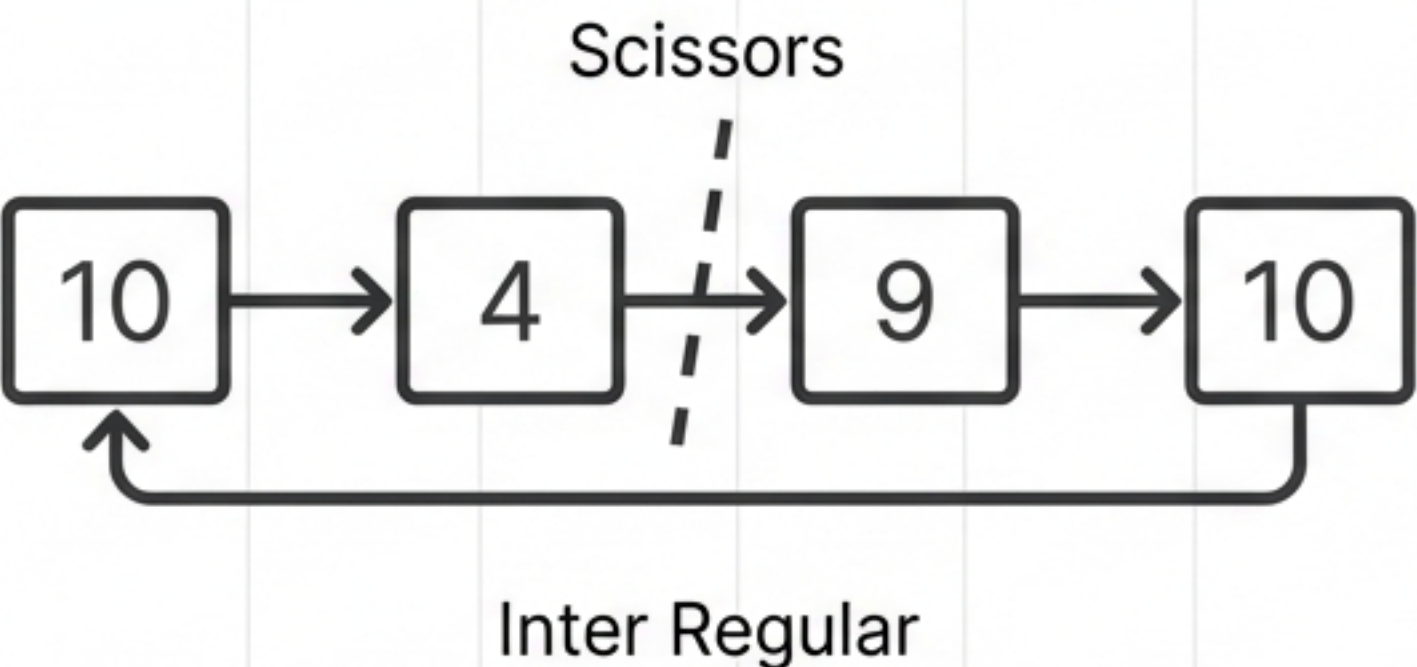
Space Efficiency

Solves complex traversal problems using constant $O(1)$ space.

Case Study 1: Split a Circular Linked List

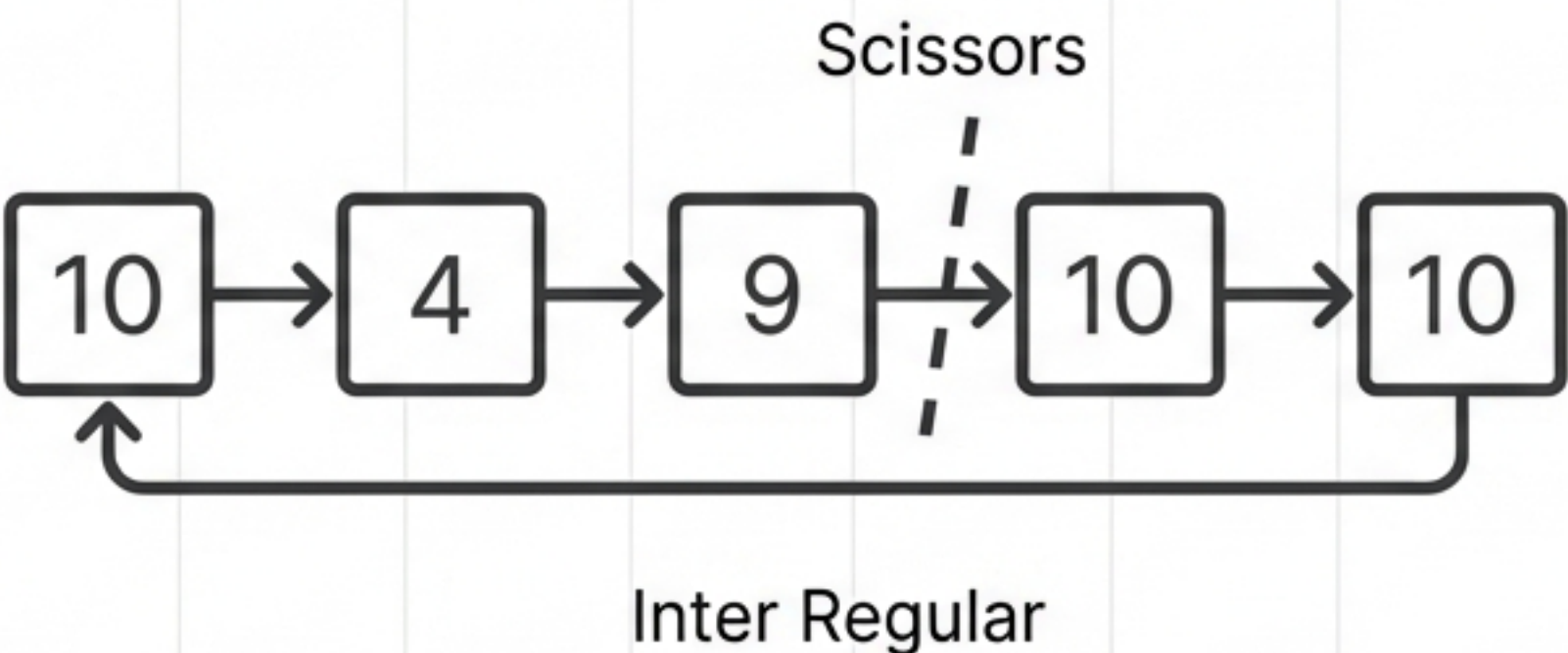
Source: [GeeksForGeeks \(Medium\)](#)

Even Nodes (Split 50/50)



Two halves of equal size.
Inter Regular

Odd Nodes (First Half +1)

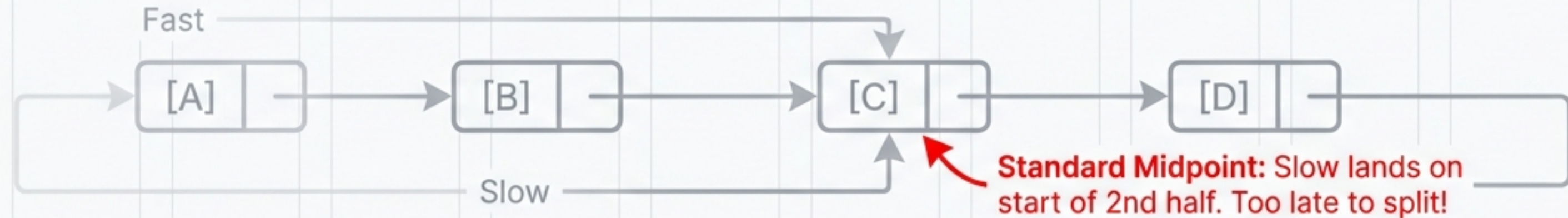


First half gets the extra node ($n/2 + 1$).
Inter Regular

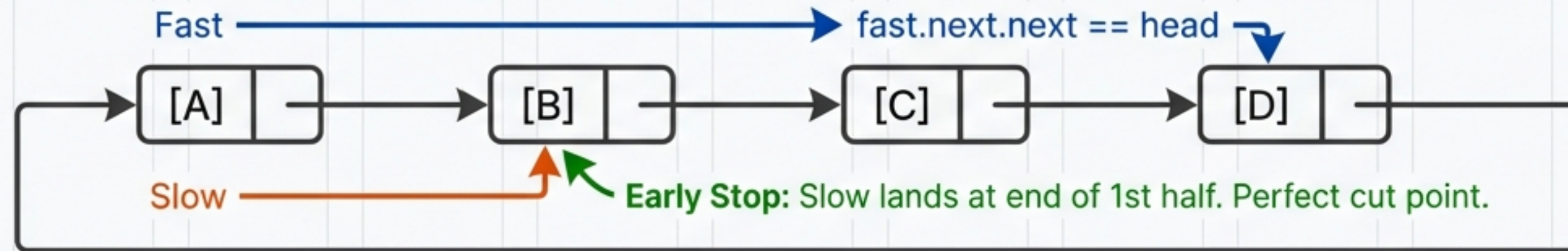
The Strategy: The 'Early Stop'

Adjusting the Fast pointer to prevent Slow from overshooting.

Standard Logic - Faded out/Greyed

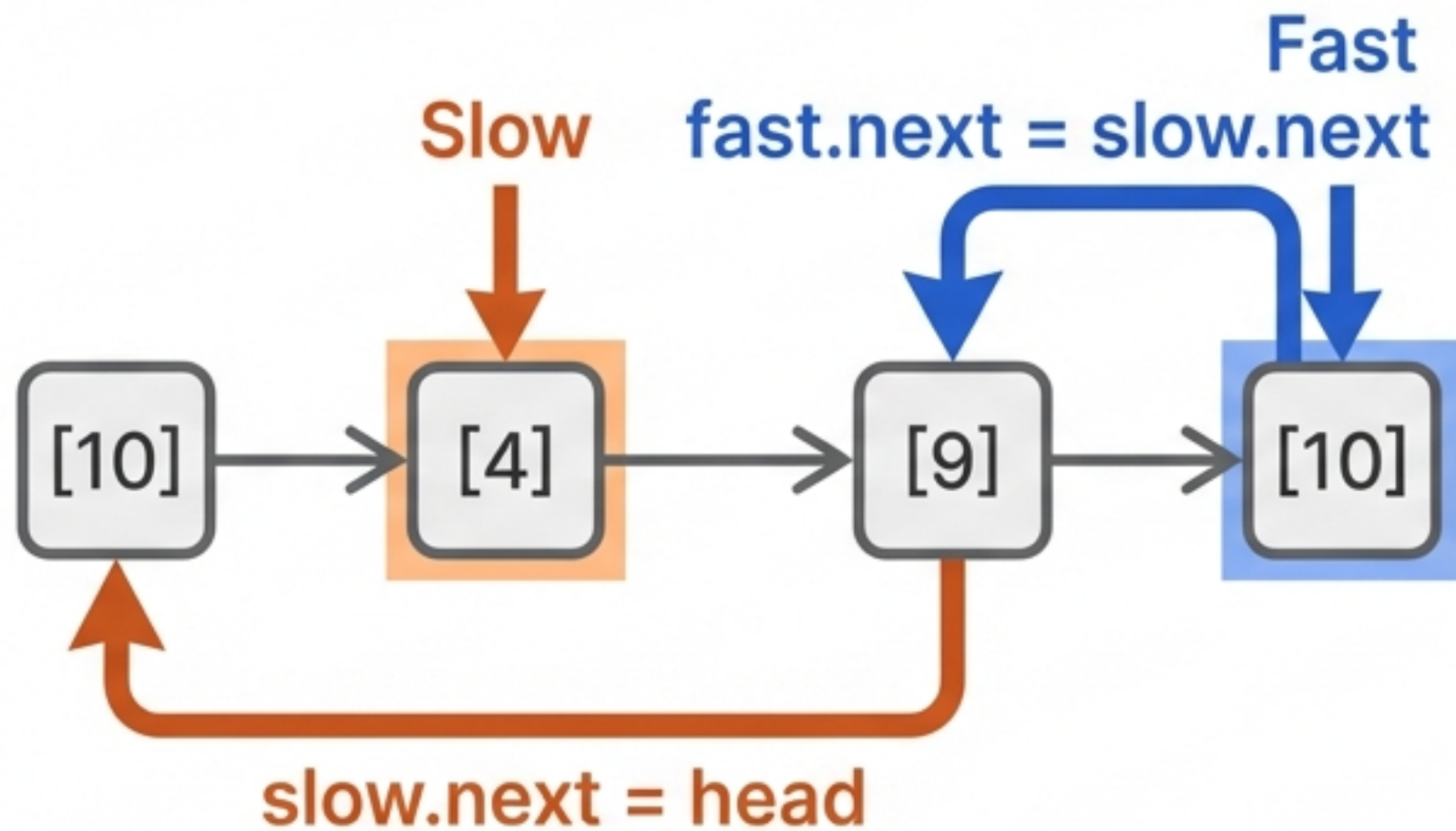


Correct Logic - Active Colors



```
while(fast.next != head && fast.next.next != head) {  
    slow = slow.next;  
    fast = fast.next.next;  
}
```


The Execution: Performing the Surgery



```
// 1. Close the second loop  
fast.next = slow.next;
```

```
// 2. Close the first loop  
slow.next = head;
```

```
// 3. Return both heads  
return new Pair(head, fast.next);
```

Case Study 2: Find the Duplicate Number

Source: LeetCode 287 (Medium)



The Constraints (The Trap)

- **Input:** Array of $n+1$ integers in range $[1, n]$.
- **CRITICAL:** Do not modify the array (No Sorting).
- **CRITICAL:** Constant $O(1)$ Space (No HashSets).

← Standard solutions violate these rules. We need a Graph approach.

The Mental Model: The 'Government Office' Analogy

Transforming an Array into a Graph.

The Array

Inter Medium, Slate Grey

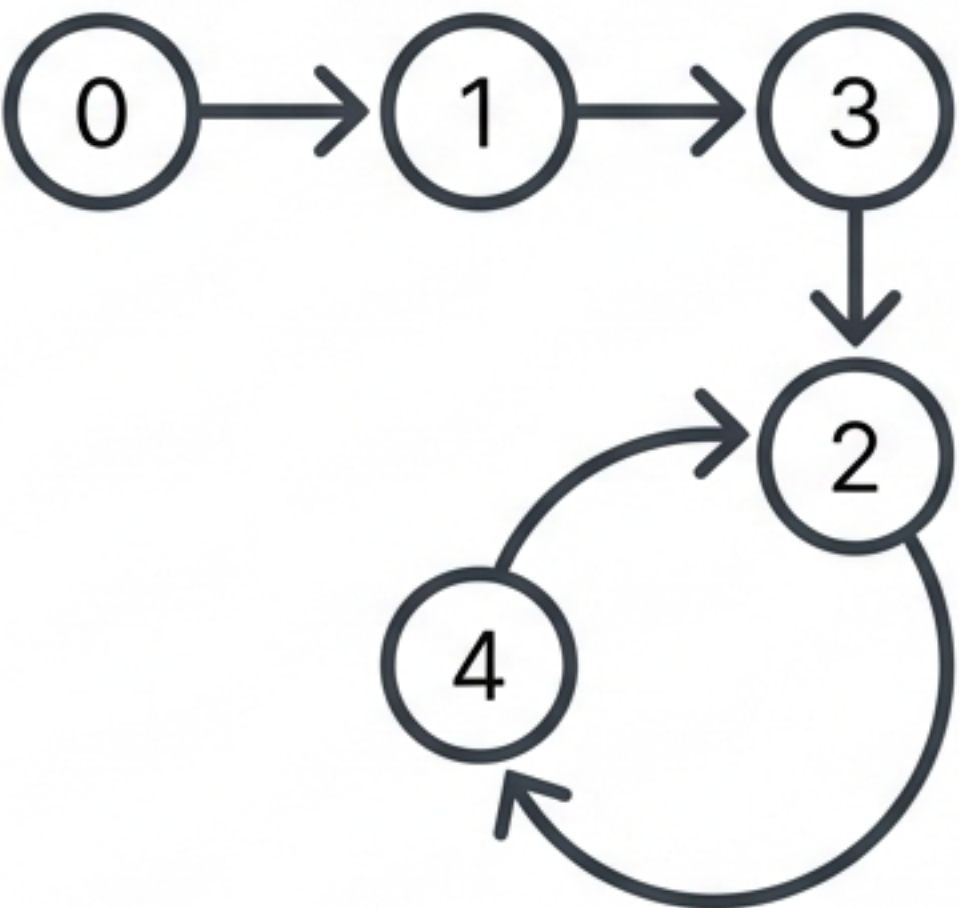
Index 0	→	Value 1
Index 1	→	Value 3
Index 2	→	Value 4
Index 3	→	Value 2
Index 4	→	Value 2

Index = Door Number | Value = Instruction
Inter Regular, Slate Grey



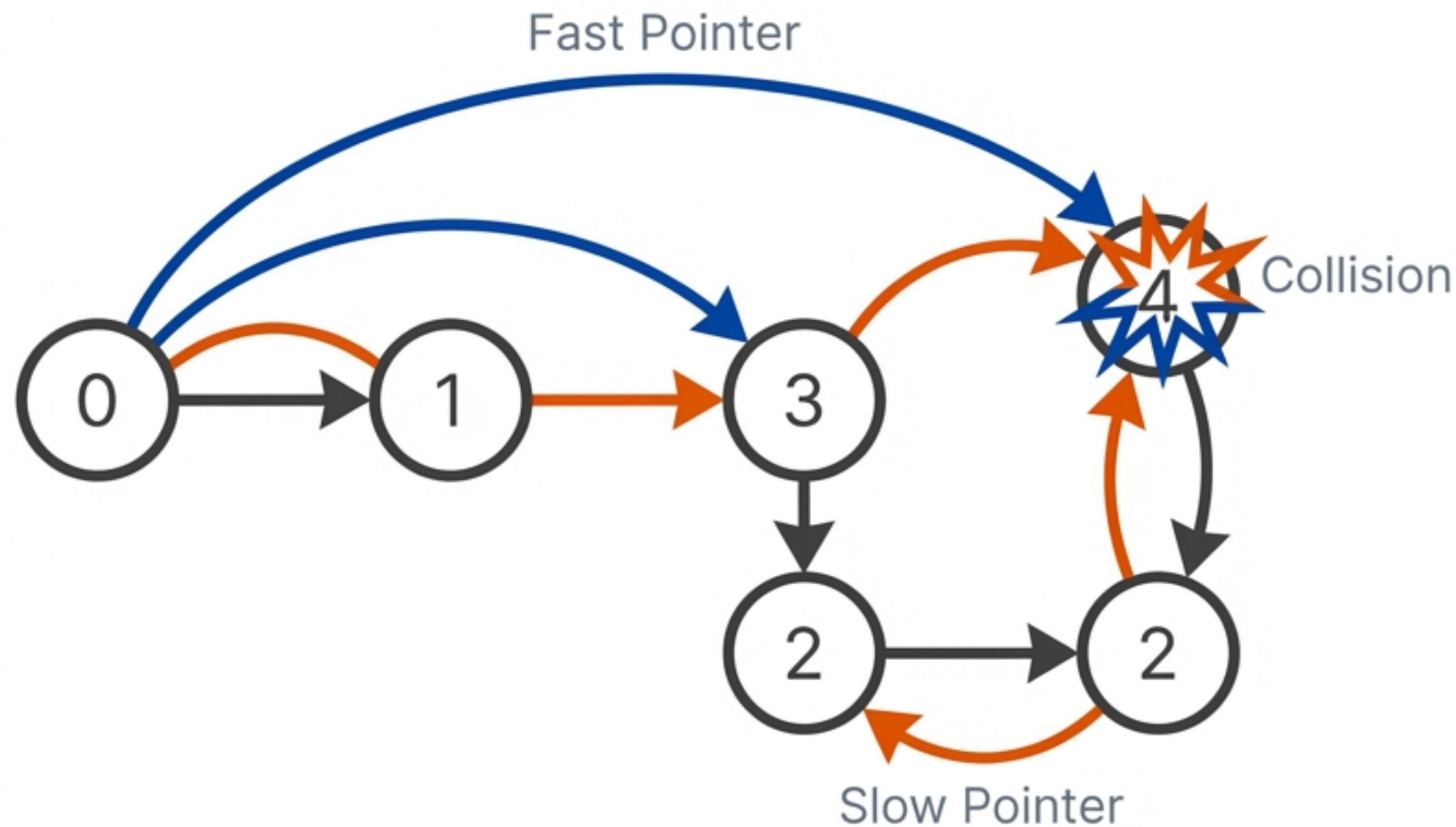
The Graph

Inter Medium, Slate Grey



A Cycle Exists because two nodes point to the same location.
Inter Regular, Slate Grey

Step 1: Cycle Detection



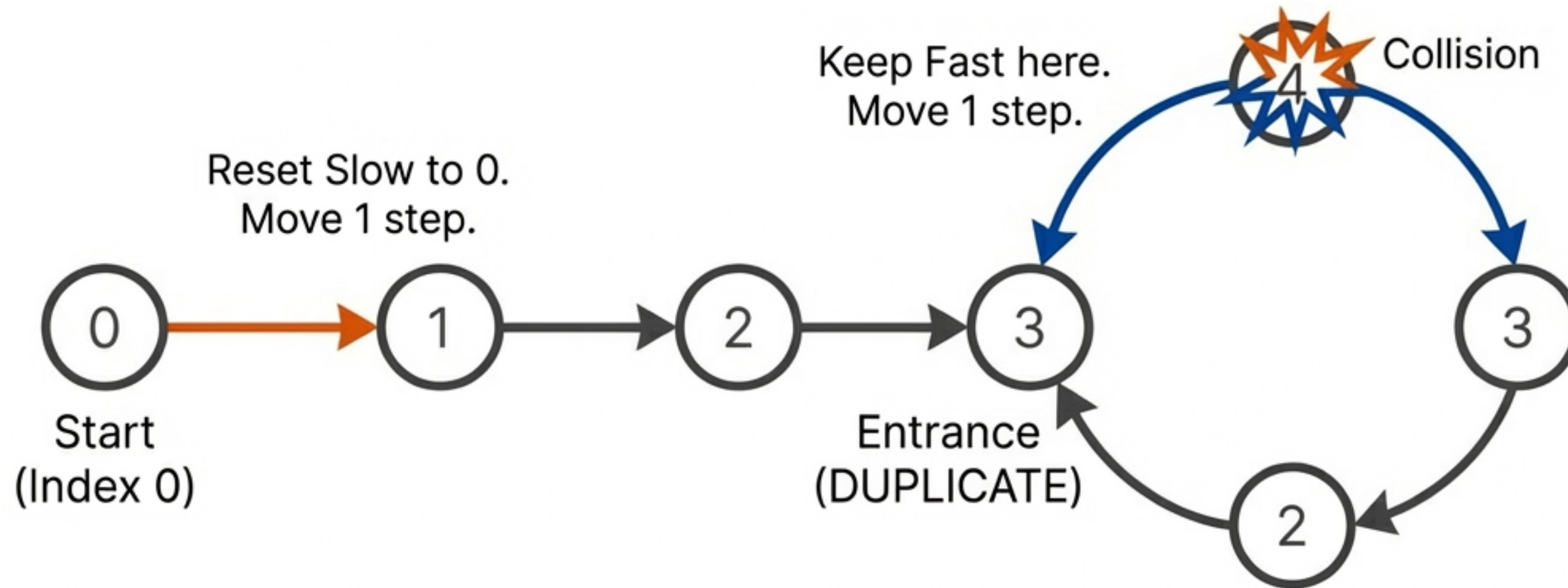
Traversal Logic

```
slow = nums[slow];  
fast = nums[nums[fast]];
```

Must use a do-while loop to handle the (0,0) start state.

Step 2: Finding the Entrance (The Duplicate)

Floyd's Cycle Finding Algorithm

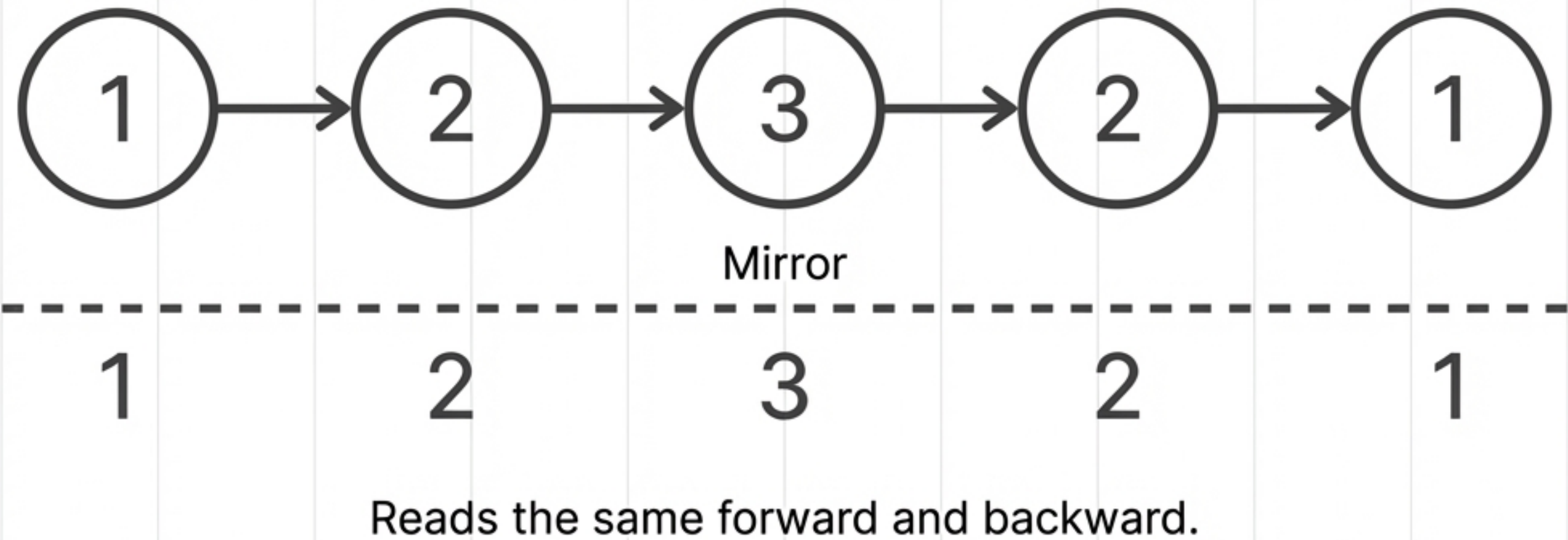


Mathematical Property: Distance from Start to Entrance == Distance from Collision to Entrance.

The meeting point is the duplicate number.

Case Study 3: Palindrome Linked List

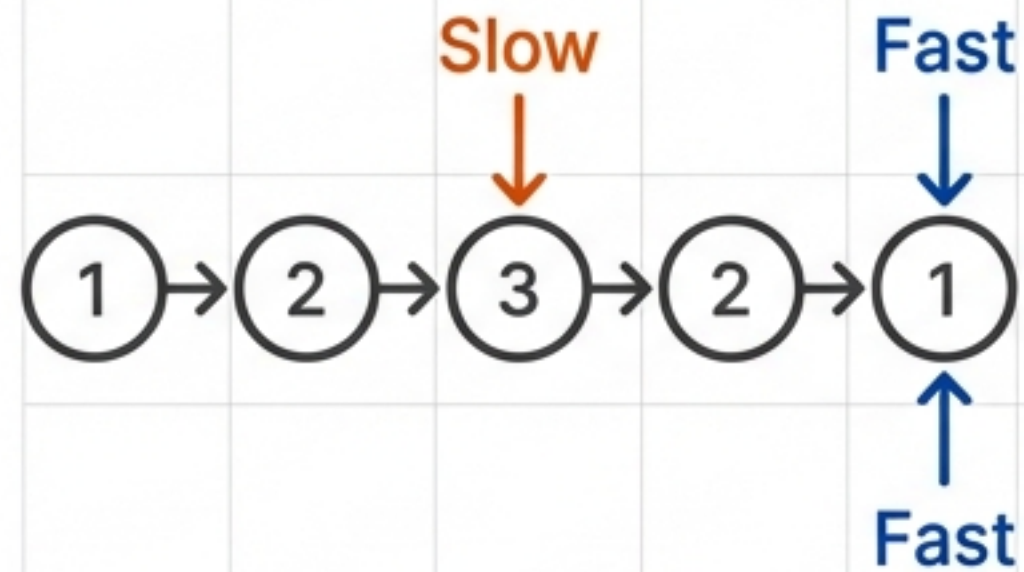
Source: LeetCode 234 (Easy/Medium)



1	Naive Approach	Copy to Array & Compare	Space: $O(N)$ (Bad)
2	Optimized Approach	Find Mid + Reverse + Compare	Space: $O(1)$ (Optimal)

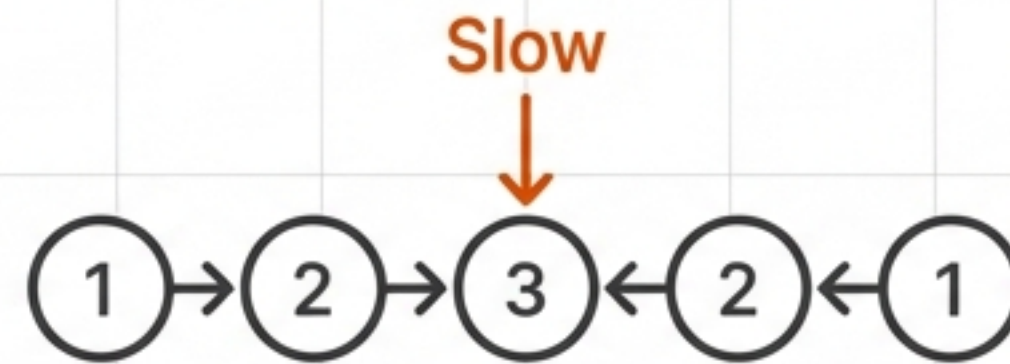
The 3-Step Solution

1. Find Middle



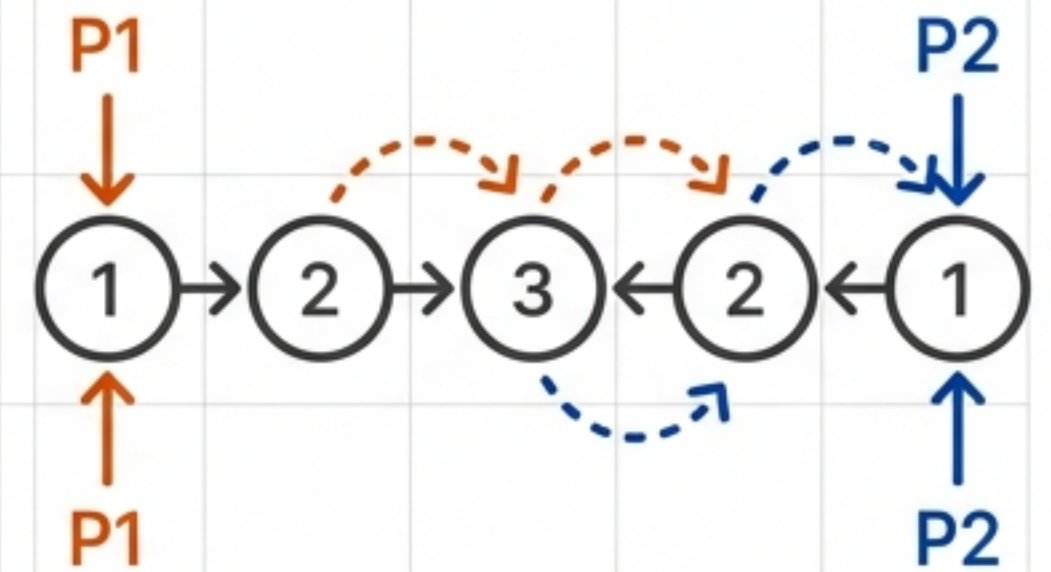
Use Fast & Slow pointers to locate the midpoint.

2. Reverse 2nd Half



In-place reversal of nodes after Slow.

3. Compare

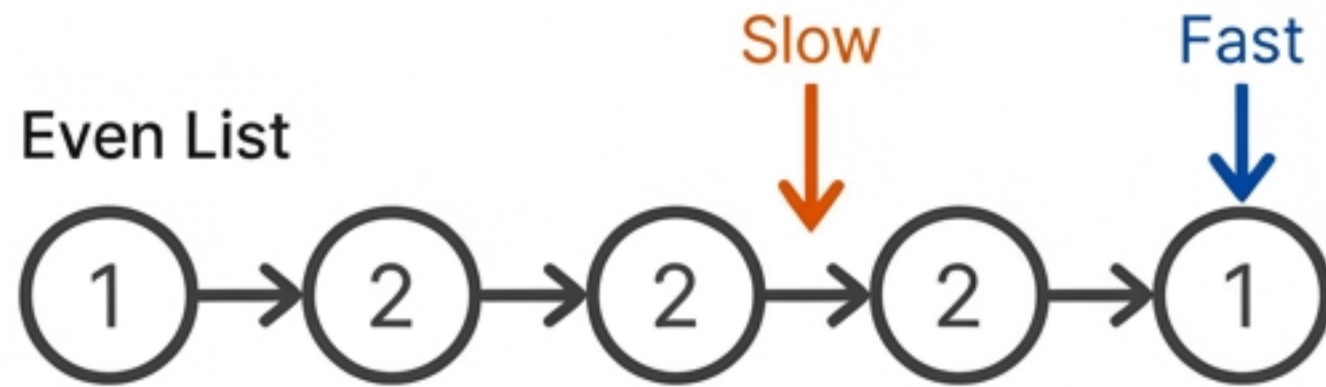


Check values step-by-step.

Edge Cases & Professionalism

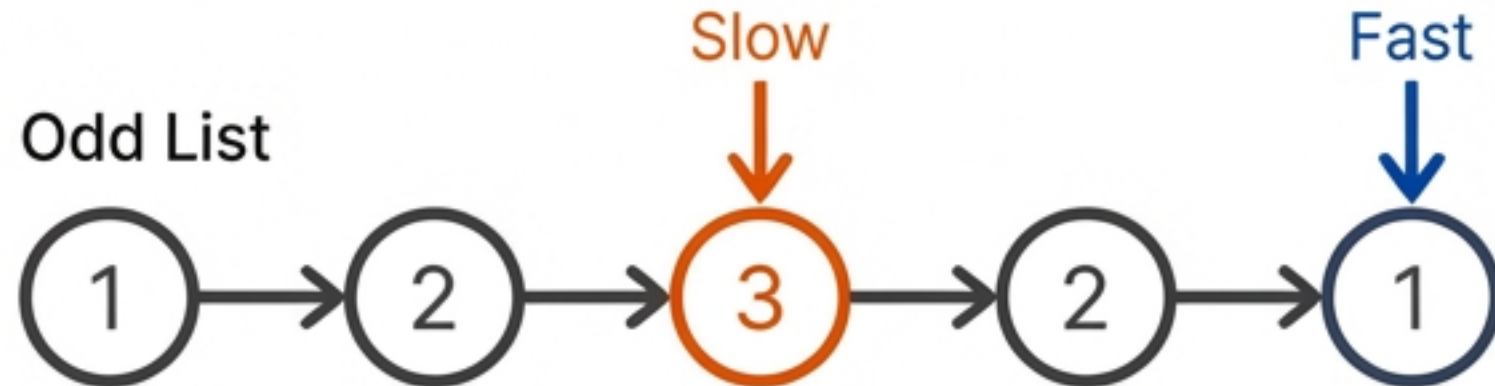
Handling Lengths

Even List



Pointers align for reversal.

Odd List



Middle node is ignored/shared during comparison.

The Professional Touch: Restoration



Modifying input is often forbidden. In a real interview, re-reverse the second half before returning.

Complexity Analysis: Time $O(N)$ | Space $O(1)$

Summary of Patterns

Problem	Pattern Role	Key Tweak
Split Circular List	Find Boundary	Fast checks ahead to stop Slow early.
Duplicate Number	Cycle Detection	Treat Array Indices as Graph Nodes.
Palindrome List	Find Midpoint	Used to enable partial Reversal.

Final Takeaways

- **Constraints Drive Design.**

(The Government Office solution exists ONLY because of the $O(1)$ space constraint.)

- **Practice the Utilities.**

(Memorize 'Reverse Linked List'. It is a building block for harder problems.)

Comment '**DONE**' to mark Day 9 complete.

Practice these 3 problems on **GeeksForGeeks** and **LeetCode**.